AgentWorkShop: An Extensible Framework for Configurable Multi-Scenario Industrial Agent Integration

Anonymous Authors

***Abstract*—Industrial agent applications must connect collaborative reasoning, production context, and heterogeneous plant interfaces while preserving a traceable path from a task proposal to its process outcome. This paper presents AgentWorkShop, an extensible framework for configuring these capabilities across industrial scenarios. Its architecture combines team and task orchestration with replaceable strategy adapters; analysis, retrieval, dataset, and model services; line–product–recipe–run context; semantic acquisition and write-control nodes; and typed interfaces for onboarding and extension. Scenario-specific signal maps, protocol bindings, objectives, and guards configure process integrations while shared collaboration and operational services remain reusable. Candidate actions enter either bounded node exploration or an operational recipe workflow, where dispatch evidence, process observations, and retained versions follow route-specific lifecycles. Evaluation exercised acquisition over five protocol families, controlled writes over four routes, strategy and controller substitutions, and four simulator process configurations. Each process had one seed-42 trajectory; all reached their declared terminal targets, two near a boundary. Nine sequential deployment-suite executions recorded 782 deterministic assertions. A single accelerated injection-molding trajectory linked four pressure trials from 45 to 65 bar, each approved by the scripted test harness, to a measured weight change from 31.28 to 32.30 g and subsequent recipe retention. The evidence supports configuration-based reuse across the exercised scenes and interfaces; all process results are simulator-based, and later checks exposed current-interface cadence limits. It does not establish physical transfer, broad plant coverage, or comparative optimization benefit.**

***Index Terms*—Industrial agents, industrial informatics, multi-agent systems, production system integration, process control, semantic data integration.**

I. Introduction

LARGE language model (LLM) agents are being applied to industrial control, factory skills, shop-floor coordination, and production scheduling [1]–[5]. These studies show that agent reasoning can be connected to plant-facing tasks, but deploying another process or line also changes signal maps, protocols, operating objectives, and production context. A reusable industrial-agent system must make those changes configurable while keeping collaboration, plant integration, and operational workflows coherent.

Injection molding illustrates this systems problem. A team member may use recent measurements and prior process knowledge to diagnose insufficient packing and propose higher holding pressure. The framework must locate the active product, recipe, and run; map the semantic quantity to a device interface; apply the operation's configured checks; and return both endpoint evidence and subsequent quality observations. The same team and operational services should remain usable when another process changes the signal map, protocol, or target criteria. A proposal, a confirmed setting, and an observed process result therefore need distinct representations, linked to the same production context.

The research question is how an extensible industrial-agent framework can connect replaceable collaboration and strategy components to production context and heterogeneous industrial I/O, while retaining route-appropriate evidence from candidate action through process observation. AgentWorkShop addresses this question through four cooperating design areas: channels, teams, tasks, and shared memory host replaceable strategy adapters; analysis, retrieval, dataset, and model services inform proposals; a line–product–recipe–run model scopes work; and semantic acquisition and write-control nodes bridge process quantities to protocol drivers. Operational profiles and capabilities determine which workflow a channel can invoke. Bounded node exploration and recipe operation then preserve their distinct dispatch, observation, and retention lifecycles. Typed SDK, REST, MCP, agent-to-agent, CLI, and console interfaces expose extension and integration points.

This paper makes three contributions:

1) It presents a modular framework architecture that joins agent collaboration and replaceable decision services with production objects, semantic industrial I/O, and external extension interfaces.
2) It describes a configuration-based integration model in which process-specific signal maps, protocol bindings, objectives, and guards reuse shared orchestration and operational services, while node exploration and recipe workflows retain route-specific evidence.
3) It evaluates protocol mappings, strategy substitutions, four simulator-based process configurations, deployment-suite behavior, and a recipe-path case. These experiments provide implementation evidence for the exercised configurations; they do not estimate physical transfer or comparative control benefit.

The remainder presents related industrial-agent and integration work, details the AgentWorkShop architecture and operational workflows, evaluates the implemented routes, and discusses evidence scope and limitations.

TABLE I
SELECTED PLANT-FACING AGENT SYSTEMS AND THEIR INTEGRATION SETTINGS. THE REPORTED EVIDENCE REFLECTS EACH SOURCE'S OWN EVALUATION SCOPE.

System	Application scope	Industrial integration	Reported evidence
MADE [1]	Operator-reviewed control-sequence generation	Structured OPC UA commands and authorization	40 control tasks; virtual and physical demonstrations
Köhle et al. [2]	Module agents and factory skills	OPC UA/MQTT with component-state tool availability	State cross-verification in a simulated six-module plant
Waseem et al. [5]	LLM-based robot-assignment scheduling	Prompt-constrained controller for a simulated line	Scheduling throughput
AgentWorkShop	Collaborative tasks, replaceable strategies, and analytic/model services	Production context and semantic DAQ/DCW nodes over multiple protocol mappings	Four configured simulator scenes; separately archived protocol, software-suite, and recipe-route evidence

II. RELATED WORK

A. Industrial Agents and Plant Integration

Industrial-agent research spans natural-language control, PLC programming, factory skills, shop-floor coordination, digital-twin support, and production scheduling [3]–[10]. Plant-facing LLM systems demonstrate complementary forms of integration, including operator-reviewed OPC UA sequences, module-scoped skills, and agent-based scheduling (Table I) [1], [2], [5]. These results motivate frameworks that can host different strategies and equipment interfaces within a common production workflow.

These systems address useful parts of the integration problem. For example, Köhle et al. restrict tool availability by component state and cross-verify states to expose silent faults in a simulated six-module plant [2]. AgentWorkShop also treats operational authority and confirmation as explicit services, within a broader architecture that coordinates collaborating agents, production objects, analysis, and process I/O. This distinction is consistent with the safety-aware decision-support perspective in [11]; the framework's workflow mechanisms do not replace process protection or establish operator effectiveness.

B. Production Context, Agent Interfaces, and Automation

MCP standardizes tool discovery and calls with request-scoped authorization [12], while ISA-88 and ISA-95 provide recipe and operations concepts [13], [14]. AgentWorkShop uses a reduced production model and run-to-run discipline [15] to connect agent tasks to configured lines, products, recipes, and runs. Software-agent constraints such as AgentSpec [16] and CaMeL's capability-based information-flow control [17] address tool behavior and data flow. The framework combines these concerns with semantic process quantities, heterogeneous drivers, timestamped observations, and operational workflows.

Electronic-record governance provides an accountability perspective: FDA guidance discusses trustworthy records and audit trails [18], while IEC 62443-3-3 specifies industrial system-security requirements [19]. Command confirmation also has telecontrol precedents, including DNP3 select-before-operate [20]. Control assurance methods address state evolution through shielding, control-barrier functions, and Simplex architectures [21]–[23]. These mechanisms complement an integration framework that must resolve which operational route applies and what evidence it returns. AgentWorkShop focuses on composing agent collaboration, production context, semantic I/O, and route-specific operational records so process configurations can change without embedding every integration decision in each strategy.

III. AGENTWORKSHOP ARCHITECTURE

AgentWorkShop composes agent collaboration, production context, analysis, industrial I/O, and operational workflows (Fig. 1). Channels organize tasks, members, shared memory, and replaceable strategy adapters; diagnostic, retrieval, dataset, and model services provide information to those strategies. A production-object model scopes the work to a line–product–recipe–run. Semantic DAQ/DCW nodes map engineering quantities to protocol drivers and return observations, while profile and capability services determine which operational route a channel may use. Typed SDK, REST, MCP, agent-to-agent, CLI, and console interfaces support integration and extension.

The design separates scenario configuration from services shared across scenarios. A deployment configures its equipment, signal maps, units, roles, setpoints, objective, and process guards; collaboration, production context, and route lifecycles use common interfaces. At runtime, a task and its production identifiers provide the context for a strategy proposal. The operational route records what was admitted, dispatched, confirmed, and observed, then returns that evidence to the task workflow. Thus an engine can be replaced without redefining a recipe or a driver's evidence contract. The project is distributed under Apache-2.0; repository links are withheld for anonymous review.

A. Semantic Nodes and Production Context

DAQ nodes expose engineering quantities with units, precision, timestamps, and quality metadata. Their contracts cover scalars, vector profiles, image frames, and mirrored manufacturing-execution-system (MES) records. MES integration supports mirrored acquisition and direct endpoint mappings. DCW nodes describe writable engineering quantities and their driver configuration. Drivers own serialization, transport, and available readback; platform services own authority and admission. A proposal can name holding pressure in bar while its driver supplies the protocol representation.

Implemented protocol families include Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, and HTTP/REST, with

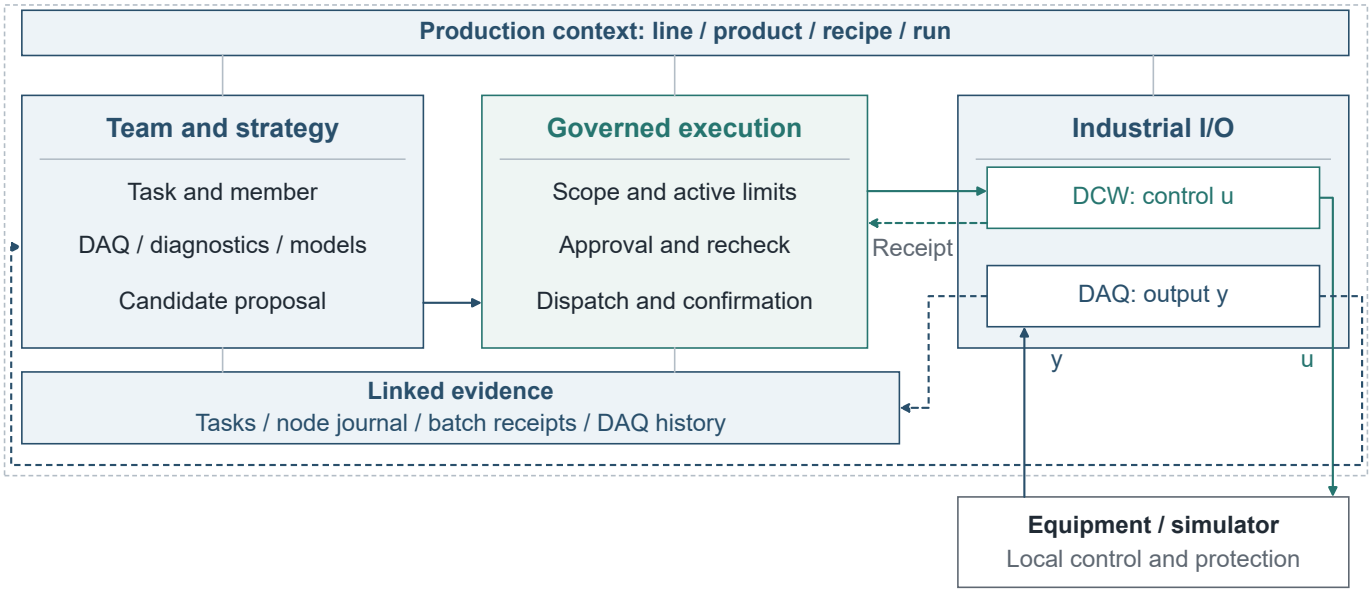


Fig. 1. AgentWorkShop component architecture. Collaboration and strategy services produce candidate operations using task, production, and analytic context. Operational services apply the selected route and retain its execution evidence. Industrial integration maps control u and acquisition y to protocol I/O; dashed return paths carry receipts and observations. Equipment protection and fast regulation remain outside the framework boundary.

MES REST mappings. Q1 exercises simulator writes over Modbus TCP, OPC UA, MQTT, and HTTP; RTU-over-TCP is acquisition-only. The admission ablation uses a separate mock driver. DAQ acquisition uses per-node cadences and a rotating sweep. Timestamped history supplies decisions, exports, and response queries independently of the agent's reasoning duration.

Lines associate equipment with an active product, recipe, and run. Recipe definitions are versioned; a running batch retains its execution context, so saving a definition and applying it are separate operations. The workflow investigates a candidate, observes its result, saves the accepted parameterization, and applies the selected version to a run. Samples and receipts retain that production context.

An onboarding blueprint declares devices, signal maps, units, roles, and initial setpoints. A differential ensure pass creates missing objects and reconciles declared configuration drift. Connecting another process changes this blueprint and its process-specific objective, guards, and proposal policy while reusing the execution services. The process missions exercise this reuse boundary.

B. Capabilities and Channel Profiles

A channel organizes a job and its collaborating members; persisted capabilities authorize industrial operations. Node bindings define readable or controlled node scope and execution mode, while recipe capabilities define an authorized production scope. Channel profiles restrict which tools a job may invoke; capabilities determine whether its member may act on the selected production objects.

In the inspected implementation, observation channels perform acquisition, analysis, and knowledge work, while Auto Modeling Lab (AML) training channels prepare datasets and models. Both have read-only plant access. AML-optimization channels invoke `optimization_explore` with control and target-scope validation. Operational channels use authorized recipe operations. Direct public `dcw_control` and `dcw_rollback` are closed; sanctioned platform workflows reach the internal governed node executor. Consequently, a written task instruction or retrieved document cannot expand the industrial tool surface.

Collaboration transfers diagnostic reports and evaluated model references to members proposing bounded interventions. Each recipient retains its own profile and capability checks: information transfer does not transfer execution authority.

C. Analysis, Models, and Candidate Proposals

Analysis and modeling supply evidence to the proposer. A merged DAQ export aligns timestamped signals and production context for industrial data diagnostics (IDD); diagnostic reports describe the observed operating regime. Retrieval-augmented generation (RAG) supplies relevant process knowledge and prior reports for interpretation. These outputs inform the strategy's choice of controls, targets, and stopping criteria. Write authority is resolved independently when the candidate reaches execution.

AML provides a separate data-to-model path. Its dataset builder selects line–product–recipe-scoped histories, cleans samples by state and range, aligns them to a sampling grid, and constructs windows within runs. Run-based partitions and training-partition normalization produce a dataset manifest, tensors, and a cleaning report. They identify the training controls, targets, features, runs, and transformations. The asynchronous production training route invokes Python, exports an Open Neural Network Exchange (ONNX) artifact, evaluates it, and applies gates before candidate registration. Registry stages distinguish candidate, shadow, production, and retired models; promotion carries its own approval workflow.

Model binding and eligibility checks govern model-assisted tools, including declared uncertainty, out-of-distribution, and physics information. Current lineage enforcement is primarily line-scoped. Each production training job retains its exported model, evaluation report, and gate decision as identifiable artifacts. Bounded exploration also operates without a model. A separately bound optimizer returns versioned recommendations and eligibility/state evidence. A recommendation grants no actuation authority; execution must enter the governed route. A trained model, candidate proposal, and industrial command remain distinct. The evaluation focuses on execution contracts and does not evaluate model-assisted control–response.

D. Two Execution Lifecycles

Figure 2 separates the current runtime routes. Bounded optimization exploration validates the writable node and target ownership, then invokes the internal governed executor. It associates a single-factor control intervention with target observations. Operational recipe work uses `recipe_trial` for a candidate override without saving a version, `recipe_update` for a new definition, `recipe_apply` for dispatch, and `recipe_rollback` for version restoration. Restoring a definition and commanding the equipment are separately evidenced operations.

Persistence follows the route. A successful changed-value node write opens a node-optimization record before the observation wait. Bounded exploration then waits, queries the target, and saves a separate exploration summary. Recipe dispatch instead creates a batch run with per-parameter receipts; its caller performs the subsequent wait, observation, and decision. Saving an accepted parameterization requires a separate recipe update. Section V identifies each experiment's tested interface.

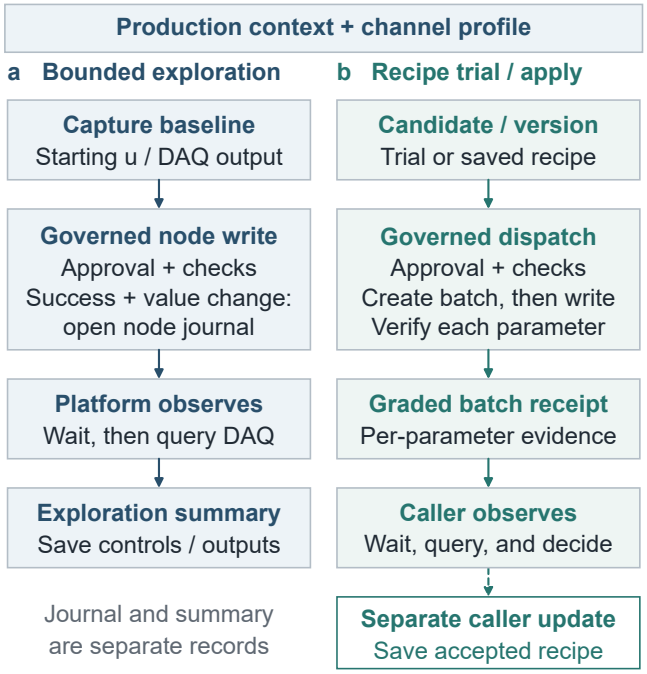


Fig. 2. Route-specific operational lifecycles. Bounded exploration opens a node record after a successful changed-value write, then waits and saves an exploration summary. Recipe dispatch returns batch and per-parameter receipts; its caller observes and decides. Retaining a recipe requires a separate update. Checks and approval follow each route's policy.

IV. Operational Workflows and Execution Contracts

AgentWorkShop connects a strategy proposal to plant-facing work through two route-specific lifecycles: bounded node exploration and operational recipe control. Both use the channel's profile and persisted capabilities, apply route-specific limits and approval, and return execution evidence to the task. Their records remain distinct: node exploration couples one changed-value intervention to a later target summary, whereas recipe work records a trial or dispatch before its caller observes the process and decides whether to retain a new version. The shared execution contract is described below.

A. Authority, Admission, and Pacing

A request identifies the actor, operation, node or recipe, and production context. The server resolves its persisted capability and channel profile; authority comes from these records. For an online node intervention, the admitted interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are baseline, active-product, and active-recipe limits. Missing optional limits impose no additional restriction. The interval in (1) is the value-admission input to the executor; an empty intersection refuses the request. The resolver identifies the excluding layer (Fig. 3a), making the decision interpretable in production terms.

Admission follows the requested operation. Recipe dispatch checks $\mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t)$, omitting the existing recipe window when applying its replacement. This lets a new recipe carry its own setpoints while preserving equipment, baseline, and product bounds. Parameter admission constrains magnitudes. The process objective and coupled-variable guards are evaluated from subsequent observations.

Timing checks regulate supervisory interventions. Online node writes have an effective minimum spacing of 60 s. Recipe dispatch uses a persistent per-recipe interval anchored to approval; trials also use a configurable per-line cadence (default 300 s). Applicable checks run at proposal or execution according to the route; the recipe-operation interval can refuse a proposal before creating an approval card. Experimental settings are stated with each trial. Stored-setpoint hold heartbeats form a separate, default-off maintenance route that reapplies the stored value without fresh admission.

B. Human Approval and Recipe Control

Manual execution bindings create an approval request carrying actor, node, candidate value, applicable interval, expiry, reasoning, and parameter bases. This structured object gives the approver the proposed operation alongside the governing context. The default timed wait expires after 180 s and refuses execution; a hold mode waits for an explicit decision. One pending request per member–node pair suppresses duplicates.

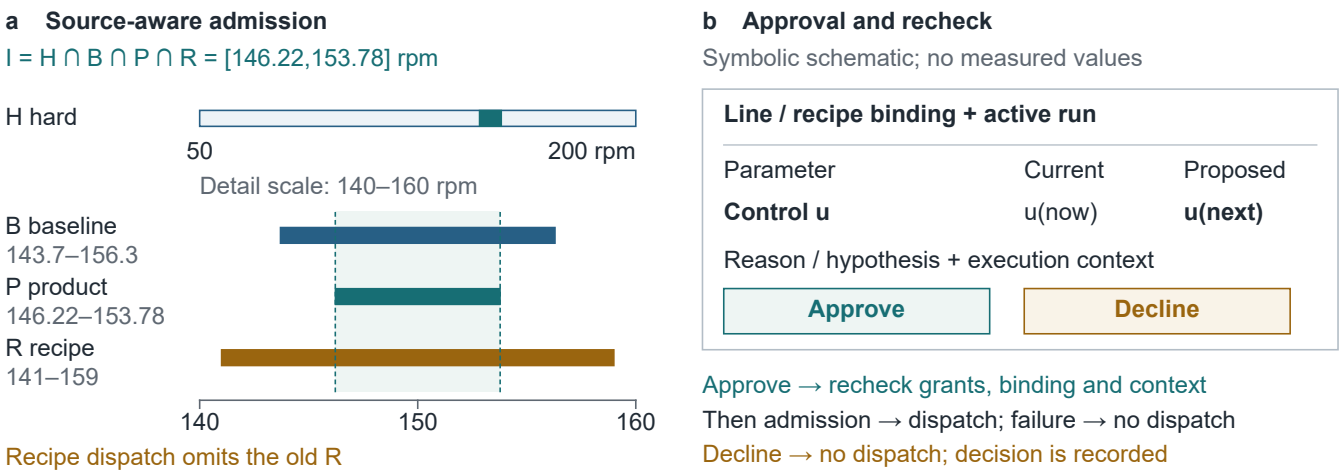


Fig. 3. Admission and approval boundaries. (a) The four-layer interval for an online node intervention; recipe dispatch omits the old recipe window. (b) Structured human approval separates the requested value and authority fields from display-only model reasoning. Approval is followed by execution checks and does not reserve equipment state.

The decision note returns to the proposing member, completing the interaction between strategy and approver.

Approval is bound to an action and followed by execution checks. The executor rechecks authorization and active limits, thereby accounting for capability or product changes during the wait. For `recipe_apply`, a changed definition requires a new proposal; an unchanged approval dispatches a copied parameter snapshot. The approved definition thus determines the values passed to batch dispatch.

A line-level recipe-dispatch master switch defaults to manual. It requires approval for otherwise approval-free recipe operations; changing it to automatic requires explicit administrator confirmation. The manual line mode also applies to internal exploration using an automatic recipe capability. Direct node bindings retain their own execution mode and admission checks. Agents use these policies but cannot change the master switch. Approval records workflow authorization; process protection retains its independently engineered control-system role.

The proposer owns the candidate and interpretation, the authenticated approver authorizes the pending action, and the verdict issuer owns the recorded decision. Receipts and operations logs identify outcomes needing operator follow-up. Deployment assigns that responsibility and the line-specific approval roles; closing a record is a lifecycle operation rather than proof of equipment recovery.

C. Graded Command Confirmation

Command evidence has two levels: the driver's acceptance ACK and the batch verifier's terminal verdict. The driver assigns `readback-verified`, `transport-ack`, or `unverified`. A matching post-command read supports the first; a transport or MES application receipt supports the second. The batch verifier assigns `verified`, `unverified`, or `failed`. It accepts an existing matching readback or performs up to three rereads, 2 s apart. A matching reread upgrades the final ACK to `readback-verified`; exhausted confirmation attempts produce an unverified verdict, and command failure produces a failed verdict. MES acceptance without a read mapping follows the unverified outcome. Write history retains the initial ACK, while the batch result stores the final ACK and terminal verdict.

Batch dispatch records each parameter's result and summarizes verified, unverified, and failed counts. This granularity preserves a verified parameter even when another parameter's command fails. A readback-verified value describes the endpoint's returned setting; target attainment is evaluated from process observations. Section V-E tests the distinction with a publish-only endpoint whose `ok=true` broker receipt is transport acceptance and whose terminal confirmation remains unverified.

D. Observation, Verdict, and Compensation

Bounded exploration couples an intervention with a target summary. It captures the starting control before asynchronous execution and averages 1-s buckets over a trailing 45-s target window. A successful changed-value write opens the node record. The exploration workflow waits 10–300 s (default 45 s) and queries another trailing 45-s window. These are query windows relative to their query times: approval can separate the baseline from actuation, and a 10-s wait can leave about 35 s of pre-write observations in the later window. The receipt and target statistics are returned separately for interpretation of this single-factor probe. The exploration summary is saved after this observation; refused or expired approval instead produces a rejected summary without actuation or post-write sampling.

The governed node lifecycle accepts attributed `keep`, `rollback`, or `uncertain` verdicts with the caller's interpretation and queried window. Compensation requests a new command under its applicable admission policy and retains its own outcome, including partial completion. In the archived experiments, a harness also compares recipe value, endpoint readback, and an acquisition-store sample. This three-way check measures agreement between representations sharing an endpoint.

A record backstop supplies monitoring while an optimization record remains open. After 120 s it polls every 30 s; three

cumulative samples outside the recipe monitoring window trigger a system-attributed rollback toward the preceding value. Its window has a monitoring role separate from the admission interval and mission target. Available samples determine breach counts; missing samples contribute no breach. A caller's keep verdict closes the record and its monitoring. When retained rollback records reach the configured limit, further actuation follows the approval policy.

Attribution joins actor, production context, receipt, verdict, and recipe/run records. The write audit deduplicates the same value on the same node within 10 s, while batch records and optimization journals retain their route-specific lifecycles. Together these records let an investigator trace the command, the evidence available to its evaluator, and the parameterization retained for subsequent work.

V. Experimental Evaluation

The evaluation asks whether shared collaboration, production-context, and industrial-I/O services can be configured across different strategies and process scenes. It leads with Q4, which pairs controls and observations for four configured processes, and Q1, which exercises acquisition and controlled-write routes over multiple protocol mappings. Q2–Q3 then examine strategy and controller substitutions; Q5 and Q6 cover deployment-suite behavior and a recipe-path lifecycle. The archives are complementary implementation evidence, not one joint current-version trial. All process experiments use repository simulators.

A. Evidence Units and Experimental Procedure

Q1–Q4 use the seed-42 integrated archive `20260921184204-10lc`, its raw scenario records, and companion ablation run `20260921181234-e1lite`. The cast-film reference preset has an author-defined objective J combining thickness conformance, defects, energy, and throughput. These historical missions use the direct-node interface. Scripted policies occupy the proposal slot except for two live engine tasks in Q2. Q3 additionally uses seeds 43–44. Q5 uses nine deployment-suite executions from October 2026; Q6 uses the October 18 recipe archive. The archives identify each tested interface and its own evidence unit. Separately, 65 channel/profile and composition checks passed in the October 20 archive and an October 24 rerun with temporary storage and mock I/O, covering approval outcomes, starting-control capture, and approved-recipe snapshot consistency. Q6 exercises the recipe route; these evidence sets are not pooled as one interface experiment.

The process missions follow six stages. First, provision the semantic nodes, their signal mappings and limits, and a baseline recipe in the active production context. Second, acquire the initial controls and target observations to establish the starting state. Third, submit a bounded candidate under the declared capability, binding, and approval policy. Fourth, retain the dispatch receipt and available readback, preserving each parameter's verification result. Fifth, wait the scenario's declared response interval and query the relevant DAQ quantities. Sixth, evaluate the observed target and guards, then continue, compensate, or stop. A retained parameterization can subsequently become a recipe version.

This ordering produces three complementary result objects. The requested control sequence u records the intervention, the endpoint confirmation records command evidence, and the target sequence y records process observations. The task verdict joins them under the declared acceptance criteria. Figures 4 and 6 display u and y together so that a target change can be inspected alongside the corresponding control history; these plots validate evidence linkage and route behavior. An evidence unit is a run or a sequential trajectory; several writes within it are sequential observations rather than independent trials.

The integrated suite has 20 phases spanning commissioning, protocol I/O, governance, agent missions, controller reuse, and concurrent processes. The deployment pipeline uses S0 prechecks, S1 APIs, S2 acquisition/MES, S3 negative paths, S4 closed-loop execution, S5 fine-tuning, S6 diagnostics, and S7 report generation. Checks are author-defined regression assertions. Reported quantities are rejected probes, per-parameter confirmation grades, recipe/readback/mirror deviations, process-target margins, and timestamp spans. With six writes per protocol line, latency is reported as the archived upper median and observed maximum rather than a tail percentile.

B. Configuration-Based Reuse Across Four Processes (Q4)

The concurrent campaign provides one seed-42 trajectory per process. Figure 4 pairs archived manipulated controls u with the corresponding response y rather than showing response alone. All four terminal attainment checks passed; two lay near a target boundary.

Injection molding used two changes: hold pressure $45 \rightarrow 58.2$ bar and hold time $6 \rightarrow 10.64$ s. Part weight rose from 31.25 to 32.17 g for a 32.5 ± 0.35 g window, only 0.02 g above its lower limit. BOPET changed casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm. Thickness fell from 28.10 to 25.66 μm for a 25.0 ± 0.7 μm target, 0.04 μm inside the upper limit. Its three changes each have a recorded keep verdict and journal identifier.

Wastewater treatment required 15 writes across several nodes. Dissolved oxygen rose from 0.21 to 3.50 mg/L, within 3.4 ± 0.6 mg/L. Some admissible blower proposals reduced compliance margin and were reverted. Terminal guards remained within their declared limits, although pH moved toward its lower bound (6.59 to 6.38; limit 6.0). Dissolved oxygen, total phosphorus, and pH are different quantities and are not concatenated into one output trajectory.

Annealing combined furnace-temperature changes with line-speed increases from 140 to 164 m/min. The recorded hardness trajectory moved from 134.52 to 99.84 HV; the separate terminal measurement was 100.08 HV, within 95 ± 6 HV. A 176 m/min attempt produced 101.3 HV and was reverted. The first temperature step has no optimization record identifier in the archive, which bounds record-level traceability. Together, the four missions exercise configuration-based process reuse

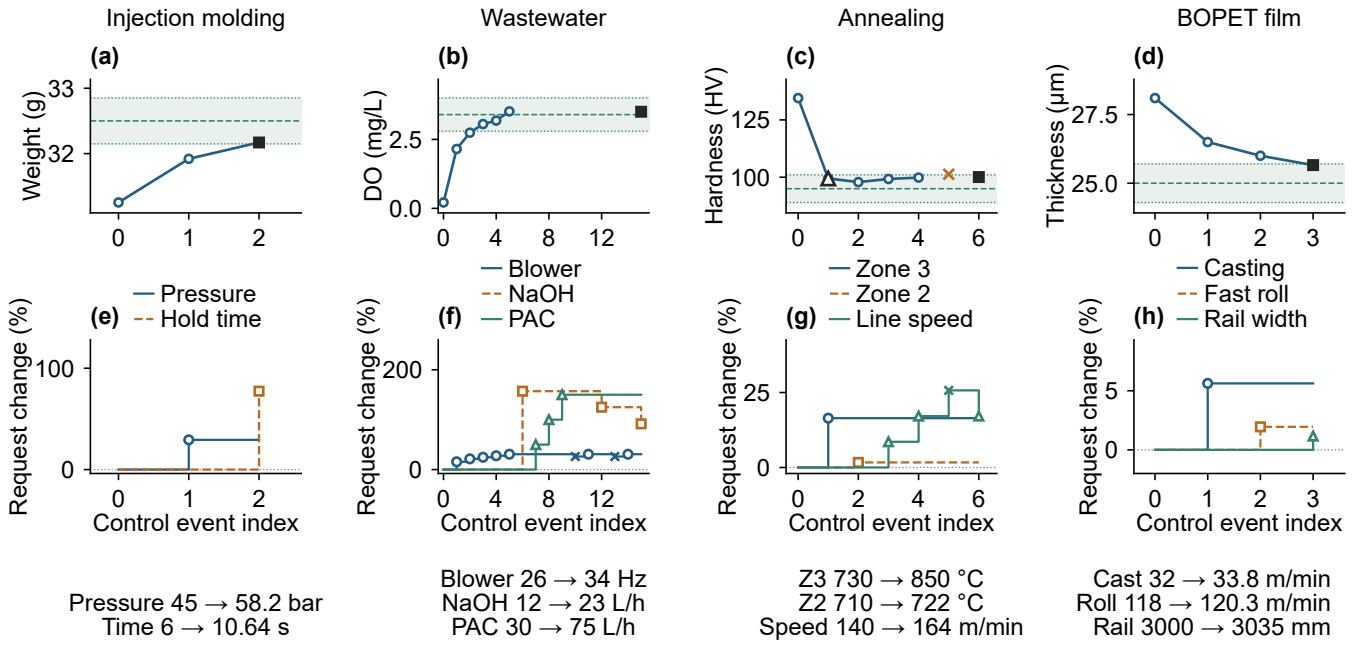


Fig. 4. Historical scripted missions (seed 42, one trajectory per process). (a–d) Weight, dissolved oxygen (DO), hardness, and film thickness with their target windows. (e–h) Requested controls as percentage change from native baselines: injection pressure 45 bar/time 6 s; wastewater blower 26 Hz/NaOH 12 L/h/PAC 30 L/h; annealing zone 3 730°C/zone 2 710°C/speed 140 m/min; BOPET casting 32 m/min/fast roll 118 m/min/rail 3000 mm. Lines connect event indices, not physical time; unchanged requested values carry forward. Crosses mark reverted candidates and squares distinct terminal observations. DO excludes pH/TP events; annealing's first step lacks an optimization record.

and retain the distinction between a candidate command, a reverted candidate, and a terminal observation.

C. Heterogeneous Protocol Integration and Route Checks (Q1)

The suite acquired 55 samples over five protocol families and issued governed writes over four; the RTU-over-TCP line was acquisition-only. Archived upper-median write latencies ranged from 19.3–47.8 ms, with observed maxima 20.1–248.9 ms. The four writable lines accepted legal writes and rejected 24/24 out-of-window probes. A configuration-only second preset added 9/9 rejected probes on three further writable lines. These results exercise the same resolver against different signal maps.

Table II isolates admission from confirmation in a mock-driver API fixture. Each arm replayed six attack classes in three deterministic repetitions (18 attacks per arm); separate boundary, legal, and timed probes are excluded from that denominator. Removing the soft-window interlock allowed six soft-window probes to execute, while the hard-range probes remained refused. Archived outcomes and journal records link the refusals and executed commands to their configured arms. Removing readback did not alter admission. Confirmation can therefore check an execution outcome, but cannot replace a pre-write gate. A frozen-measurement drill further separated command and process evidence: the setpoint and readback agreed, yet the monitoring quantity remained outside its window and triggered backstop rollback after 130.2 s. A manual-binding drill suspended execution until harness-issued approval (461 ms mediation span).

TABLE II
MOCK-DRIVER API ABLATION: SIX ATTACK CLASSES, THREE DETERMINISTIC PROPERTY-TEST REPETITIONS PER ARM (18 ATTACKS, EXCLUDING SEPARATE BOUNDARY PROBES). FOUR HARD-RANGE CLASSES ARE REFUSED IN EVERY ARM; TWO SOFT-WINDOW CLASSES CHANGE OUTCOME.

Arm	Soft window	Readback	Refused	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6
ungated	—	—	12/18	6

D. Strategy and Controller Substitution (Q2–Q3)

The archived team workflow linked a parent task, a bound worker, a command, a verdict, and its journal. On the single-node cast-film mission, the policy changed the heater from 200 to 204.704 °C; the observation settled at 204.70 °C within the declared target. The task reached `COMPLETED`. The BOPET mission expanded this workflow to nine devices and 49 signals, with three manipulated nodes (Section V-B).

Two live engine tasks (omp adapter, glm-5.3-flash) occupied the same proposal slot on a scratch line. The archive records a bounded write, a goal-driven descent to 51.7 μm for a $52.0 \pm 0.8\,\mu$m target, and a recipe save. The original contemporaneous omp logs and checked run outcomes trace proposal, governed action, observations, and retained recipe. The reported 51.7 μm is the run-summary terminal value; the log's 52.02 μm window mean is a different aggregation.

A deterministic controller used the dimensionless cast-film objective

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \tag{2}$$

TABLE III
DEPLOYMENT-SUITE ASSERTIONS ACROSS NINE SEQUENTIAL EXECUTIONS; CHECKS ARE DETERMINISTIC ASSERTIONS, NOT INDEPENDENT PLANT SAMPLES.

Pass	Test surface	Runs	Checks
1	baseline S0–S7	3	237
2	+ grading and master switch	3	273
3	hardened regression surface	3	272
Total	zero failed checks	9	782

whose components score thickness conformance, product quality, and control-based energy and throughput proxies. The latter two terms use screw speed and line speed, rather than independently metered energy and production output. The functions in (2), constraints, grid, and replay settings are specified in the accompanying `REPRODUCIBILITY.md`. It made two governed writes per seed (42–44), without rejection. It raised J from approximately 67.2 to $J/J^* = 0.965$–0.972, with $J^* = 89.894$ from an admissible setpoint grid. This finite restricted grid supplies a reference normalization, not a global continuous optimum. The result links the controller's native objective to admissible commands and endpoint evidence, exercising a conventional strategy through the same execution services.

E. Deployment Composition and Confirmation Semantics (Q5)

The deployment instance contained 34 provisioned lines, 111–113 channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 bindings, with 17 lines running. These objects coexisted on one simulator-backed host. Nine pipeline executions passed 782 checks (Table III).

The injection benchmark followed observation, proposal, pacing gate, approval, batch dispatch, and recipe/readback/mirror comparison. The three sources agreed at the endpoint's recording resolution (maximum deviation 0.0 bar; tolerances 0.05 bar for device readback and 0.51 bar for the bucketed mirror). This verifies setpoint consistency rather than process convergence. In the illustrated S4 run, the trial cadence refused the initial proposal before approval; the reproposal occurred 69 s later and observation-to-verification took 96.105 s. S5's wait-to-restoration span was approximately 66 s (Fig. 5); the archived process result remained `converged=false`. Across the campaign, approval-to-dispatch spans were 6–9 ms. These spans measure server/harness execution between the recorded events.

The seven-leg fault suite tested the interpretation of receipts. A mock acceptance was promoted by reread; a broker outage produced one failed parameter while its healthy sibling verified; and a publish-only endpoint returned a broker receipt yet remained unverified after three attempts. Two parameters were promoted from transport acknowledgment to readback verification. Manual-to-automatic mode changes required confirmation, and a manual-mode recipe release queued approval before six parameters verified. API and documentation checks completed the suite. MES application-receipt semantics are covered by route tests, whereas the campaign used MES only in read mode. The suite therefore checks that receipt grades follow the available endpoint evidence under each deliberately injected condition.

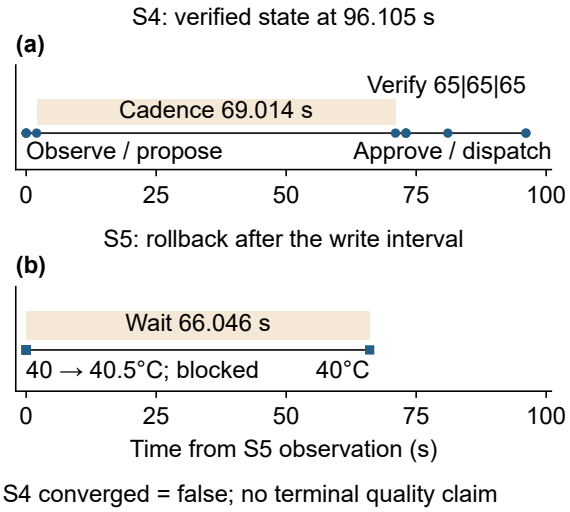


Fig. 5. Archived governance timing from event timestamps. (a) S4 takes 96.105 s from observation to set/device/mirror agreement at 65 bar, including a 69.014-s cadence wait. (b) S5 starts 96.129 s after the S4 observation, applies $40 \rightarrow 40.5\,^{\circ}$C, blocks a further write, waits 66.046 s, and restores 40°C. Each row has its own elapsed-time origin. Scripted approval and command verification do not establish quality convergence; the S4 verdict is `converged=false`.

F. Operational Recipe Workflow Case Study (Q6)

The October 18 recipe archive (R6 in the accompanying reproducibility notes) pairs recipe-path controls, receipts, and process observations. The replay entry is `scripts/e2e-injection-recipe-closedloop.mjs`. The setup used OPC UA setpoints and HTTP quality acquisition against the injection simulator. A live omp/deepseek-flash worker separately diagnosed insufficient packing. The archived intervention sequence and approvals were scripted, allowing the candidate, receipt, wait, observation, and recipe-update sequence to be inspected explicitly.

Starting from 45 bar and a fixed 6-s hold time, the script proposed 5-bar pressure increments. Each step called `recipe_trial`, answered the pending approval card, and retained the batch receipt. After 65 s, it queried platform control-node values and DAQ outputs. It stopped when weight was in $[32.15, 32.85]$ g, flash was at most 0.4%, and sink mark was at most 1.5%, or after at most six pressure proposals. The replay also defines overshoot compensation and a hold-time fallback. The archived successful run used four pressure proposals and no hold-time change. The harness set `AW_TRIAL_INTERVAL_MS=0`, disabling the per-line trial cadence, and issued `emergency=true` trials to bypass the separate recipe-operation interval. Approval, admission, and command confirmation remained active. This configuration tests the execution chain with experimental pacing. An earlier archived attempt exercised the default 300-s cadence: it refused a premature trial and the target was not attained in that attempt.

Figure 6 shows the complete baseline-to-terminal sequence. Pressure advanced $45 \rightarrow 50 \rightarrow 55 \rightarrow 60 \rightarrow 65$ bar; weight increased $31.28 \rightarrow 31.49 \rightarrow 31.74 \rightarrow 32.05 \rightarrow 32.30$ g, and

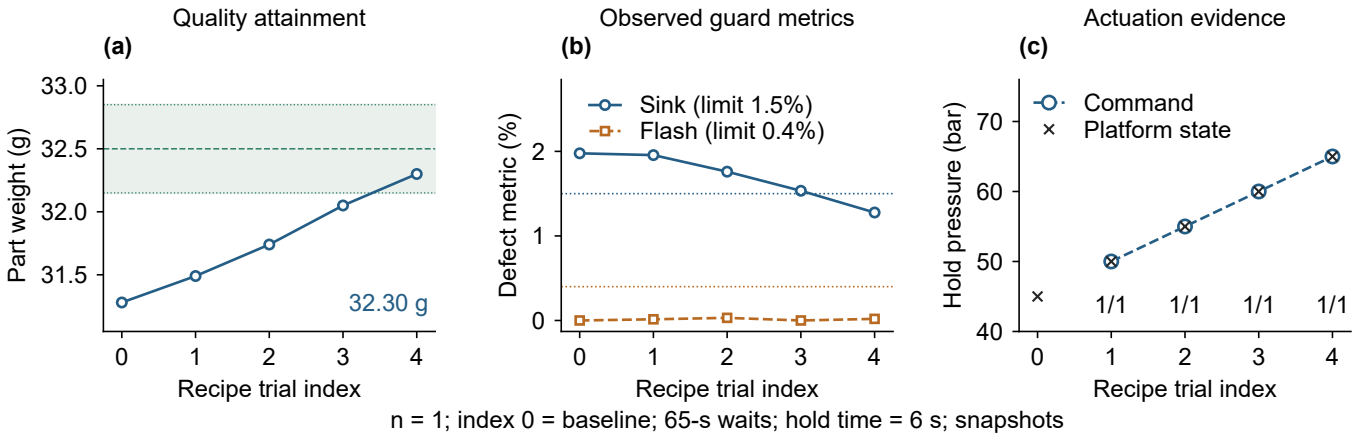


Fig. 6. October 18 recipe-path injection case study ($n = 1$ simulator trajectory). (a) Weight with the 32.15–32.85-g band and 32.5-g center. (b) Sink and flash snapshots with 1.5% and 0.4% limits. (c) Requested pressure and platform-node state after each 65-s wait; overlapping symbols indicate equality, and each trial receipt reports 1/1 parameter verified. Hold time remains 6 s. Points are discrete snapshots, not physical PLC traces; the harness supplied approvals.

sink mark decreased from 1.977% to 1.277%. Final flash was 0.019%. Each trial receipt reported one verified parameter out of one. The displayed controls are platform-node snapshots, while the receipt supplies the archived verification grade. After the final check, `recipe_update` saved version 2 with 65 bar/6 s. The recipe log contains four trials and the update, while the optimization journal is empty, as expected for this lifecycle.

The paired plot records the stopping rule: at 60 bar, the 32.05-g observation remained below the 32.15-g acceptance limit; at 65 bar, all three quality criteria passed. Pressure was the sole changed control. The separate recipe update retains the accepted setting. Its rationale contains different quality figures without a linked raw sample; the plot uses the archived final snapshot.

The quality assertion allowed a missing melt-temperature value, and the freshness check used the newest timestamp among available sensors. The reported pass therefore covers the three observed quality quantities; temperature and individual-sensor freshness remain outside that assertion. The duplicate archive adds no repeat. A separate later check started at 65 bar, 32.29 g, and 1.385% sink mark; one 70-bar trial ended at 32.55 g and 0.976%. It is a new initial state and is not appended to the plotted trace. Q6 connects baseline evidence, approved control changes, observed quality, and the retained recipe through one inspectable operational loop.

October 23 boundary checks distinguish historical replay from current operation. The public interface refused all eight direct-node attempts (zero writes). A separate recipe check verified the first 45-to-50-bar trial but refused requests at 66 and 132 s under the default 300-s cadence. Weight remained 31.46 g and no recipe update was saved. These failed checks are not repetitions of the accelerated October 18 trajectory; their full outcomes are retained in the reproducibility notes.

VI. Engineering Interpretation

The evaluated contribution is configuration-based reuse across industrial agent settings. Q4 exercises four process configurations, each with one seed-42 simulator trajectory; all met their declared terminal checks, two near a boundary. Q1 covers acquisition across five protocol families and writes on four (RTU-over-TCP was acquisition-only). Q2 contains two live engine tasks; Q3 uses three seeds and a restricted setpoint grid; Q5 records 782 deterministic assertions across nine software-suite executions; Q6 is one accelerated recipe trajectory. Together, these archives exercise strategy, I/O, scene configuration, and workflow integration, but do not compare optimization performance.

Authorization, admission, approval, and graded confirmation are cross-cutting operational services within the framework. Their records connect an actor and production scope to a command receipt and process observation. They do not measure operator comprehension, complacency, or responsibility allocation [24]–[26]. The execution boundary is independent of prompts and retrieved documents [27], [28]; deployment still requires credential management, endpoint security, and IT/OT separation [19]. It complements rather than replaces independently engineered PLC interlocks and safety-instrumented systems; IEC 61511 governs the latter's process-safety lifecycle [29].

Experimental scope. Process evidence is simulator-based and approvals were scripted. Q1–Q4 use a historical direct-node interface; Q5 comprises software assertions, not process samples; Q6 is one October 18 recipe trajectory with cadence overrides. The archives do not form a joint current-version end-to-end study, and a complete current exploration-to-process-response test is not archived. On October 23, the current public interface refused all eight historical direct-node attempts. A separate current recipe check verified only the 45-to-50-bar trial, refused later requests at 66 and 132 s under the default cadence, reached no target, saved no recipe, and failed its restoration assertion. These checks expose interface boundaries; they do not repeat the accelerated case. The evidence does not estimate physical transfer, general plant coverage, hard real-time behavior, fault frequency, human performance, or comparative benefit. Repeated matched runs, a current-route end-to-end study, and physical evaluation are needed; process benchmarks such as Tennessee Eastman [30] offer one further setting.

VII. Conclusion

AgentWorkShop combines collaborative tasks and replaceable strategy services with production context, semantic industrial I/O, and distinct node and recipe workflows. Across the tested simulator scenes and protocol routes, the implementation exercised configuration-based reuse and route-specific evidence. These results clarify what the framework connects and where its current evidence stops; they do not establish general industrial coverage, physical transfer, or comparative control benefit.

Data availability and AI assistance. Run archives, original Q2 engine logs, replay entries, configuration notes, and claim-to-source mappings accompany the separate anonymous experimental evidence bundle. The manuscript source package contains plotting sources and figure mappings. An OpenAI language model assisted with manuscript organization, language revision, and plotting-code preparation. OpenAI image generation drafted the compositions of Figs. 1 and 2; the final artwork was redrawn as editable Python-generated vectors and checked against the implementation. The authors remain responsible for the final submission.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[3] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[4] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[5] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[6] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[7] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[8] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[12] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[13] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[14] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[15] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[16] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[17] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[18] U.S. Food and Drug Administration, "Part 11, electronic records; electronic signatures—scope and application," Guidance for Industry, 2003.

[19] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[20] IEEE, "IEEE std 1815-2012: Standard for electric power systems communications—distributed network protocol (DNP3)," Standard, 2012.

[21] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[22] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi: 10.1109/TAC.2016.2638961.

[23] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.

[24] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[25] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[26] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[29] IEC, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.

[30] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993, doi: 10.1016/0098-1354(93)80018-I.